

Optimal Control

Introduction to Python

Marco Falotico

Giuseppe Notarstefano

Department of Electrical, Electronic, and Information Engineering
Alma Mater Studiorum Università di Bologna
giuseppe.notarstefano@unibo.it

The present slides are for internal use of the course
Optimal Control @ University of Bologna

What is Python?

- High-level, general-purpose programming language
- Designed for readability, with simple syntax
- Popular for web development, data analysis, automation, scientific computing, and artificial intelligence
- Cross-platform: Available on all major operating systems



Operating System Requirements

The Python distribution is Python 3.x
Latest stable Release is 3.13.0 in Oct.7 2024

The lab computers provide the necessary environment
create your account at <https://infoy.ing.unibo.it>

Why Python?

- It works on different platforms (Windows, Mac, Linux, Raspberry Pi...)
- It has a simple syntax, similar to the English language, and has a compact syntax
- Python runs on an interpreter system: the code can be executed as soon as it is written (prototyping can be very quick)
- Python can be treated in
 - ▶ a procedural way
 - ▶ an object-oriented way
 - ▶ a functional way



Integrated Development Environment (IDE) for Python

Get a text editor/IDE

- Visual Studio Code
- PyCharm (IDE)
- Notepad++/gedit
- ...



Python Distribution and Virtual Environment

The official website is <https://www.python.org/>

The complete documentation about Python can be found at <https://docs.python.org/3/>

Download <https://www.python.org/downloads/>

- On Windows try also Anaconda
- On MacOS try also Homebrew

We will be using the “Virtual Environment”

- Prevents version conflicts
- Lighter than a virtual machine
- Allows to install several versions on a single machine



Creating a Virtual Environment - In Terminal (Windows)

On Windows:

- To create a Virtual Environment

```
python -m venv C:\path\to\new\virtual\  
environment
```

- To activate the Virtual Environment:

```
# Unlock script execution  
set-executionpolicy remotesigned  
# Activate Environment  
C:\path\to\new\virtual\environment\Scripts\  
Activate.ps1
```

- To deactivate, type deactivate



Creating a Virtual Environment - In Terminal (Linux or Mac)

On Linux or Mac:

- To create a Virtual Environment

```
python -m venv /path/to/new/virtual/  
environment
```

- To activate the Virtual Environment:

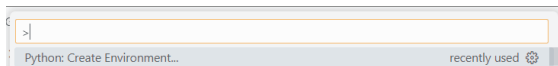
```
source /path/to/new/virtual/environment/bin/  
activate
```

- To deactivate, type deactivate

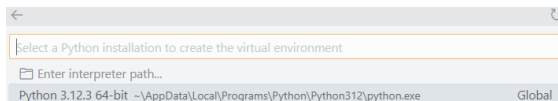


Creating a Virtual Environment - In Visual Studio Code

Enter settings with {Ctrl + Shift + P} and choose Venv or Conda



Create a Virtual Environment with a chosen interpreter
Note: Each venv can have different interpreters



Visual Studio Code also allows you to choose previously created environments

Install Additional Packages

A package manager for Python packages is called PIP

Example of usage for the package “numpy” is

```
pip install numpy
```

Then we can use it in Python programs adding the line

```
import numpy
```



Hello, World!

Create a file named

```
helloworld.py
```

Containing only the line

```
print('Hello, world!')
```



Try to make it work! (Better if within a virtual environment)

Interpreted vs Compiled Languages

- **Interpreted Language:**

- ▶ Python code is executed line-by-line by an interpreter
 - ▶ Interactive terminal code execution can be performed

```
>> x = 10
>> x * 2
>> print(_)
20 #variable "_" stores the output of the latest operation
```

- ▶ Allows runtime debugging, useful for testing and development
- ▶ Drawback: Slower execution compared to compiled languages (e.g., C, Java)

- **Compiled Language:**

- ▶ Converts code to machine-level before execution
- ▶ Faster execution but requires a compilation step before running

Dynamic Typing

- Python uses **Dynamic Typing**:
 - ▶ No need to declare variable types explicitly
 - ▶ Type of a variable is determined at runtime based on the assigned value
- Example:

```
x = 10          # int
x = "Hello"     # now a string
```

- More flexibility, but type errors may occur at runtime
- **Note:** Dynamic typing may have unpredictable consequences

Dynamic Typing - Example

```
import numpy as np

# Initialize a NumPy array (vector) with integers
vector = np.array([1, 2, 3, 4])

# Check the data type of the array
print("Original vector:", vector)
print("Original data type:", vector.dtype)

Original vector: [1 2 3 4]
Original data type: int64
```

Dynamic Typing - Example

```
# Try to assign a floating-point value to an element
vector[0] = 3.14

# Check the vector after assignment
print("Modified vector:", vector)
print("Data type after modification:", vector.dtype)

Modified vector: [3 2 3 4]
Data type after modification: int64
```

- Numpy array was initialized with integer entries
- Python enforced integer type on the vector
- New assignments are truncated to the integer part

Always verify to have the correct type for your array!

Memory Management

- Python manages memory automatically
 - ▶ No memory allocation is needed (differently from C language)
 - ▶ Memory freed automatically with **Garbage Collection**
- Unused objects are automatically cleaned up, freeing up memory
- **Reference counting** cleans up memory whenever an object is not referenced
- **Cyclic Garbage Collection** deallocates memory for cyclic references

Python Data Types (Built-in)

- Python supports several built-in data types:
 - ▶ `int` – Integer values (e.g., 5, -3)
 - ▶ `float` – Floating-point values (e.g., 3.14, -2.7)
 - ▶ `str` – Strings (e.g., "Hello, World!")
 - ▶ `bool` – Boolean values (True, False)
- Variables are created by assignment.

```
x = 10          # int
y = 3.14        # float
name = "Argo"   # string
```

Python Data Types (Collections)

- **Collections** in Python:

- ▶ list – Ordered, mutable collection (e.g., [1, 2, 3])
- ▶ tuple – Ordered, immutable collection (e.g., (1, 2, 3))
- ▶ dict – Key-value pairs (e.g., {"a": 1, "b": 2})

- Example:

```
my_list = [1, 2, 3]
my_tuple = (4, 5, 6)
my_dict = {"key": "value"}
```

Everything is an Object

- In Python everything is an object
- Variables, built-in types, functions, classes are objects
- High flexibility in memory allocation
- Metadata Overhead

```
x = 10

# Check if 'x' is an instance of the 'int' class
print(isinstance(x, int)) # Output: True

# Integers have methods because they are objects
print(x.bit_length()) # Output: 4
```

Everything is an Object - Examples

To reduce **Metadata Overhead**, simple objects are reused. This optimizes memory usage but can cause some issues

- Small integers are cached and reused

```
x = 10
y = 10
print(x is y)    # Output: True (same memory location)
```

- Large integers are not

```
a = 1000
b = 1000
print(a is b)    # Output: False (different memory locations)
```

Everything is an Object - Examples

Lists can face more severe issues

```
list1 = [1, 2, 3]
list2 = list1 # list2 is just another reference to list1

# Since both point to the same list object, modifying one affects the
# other
list1.append(4)

print(list1) # Output: [1, 2, 3, 4]
print(list2) # Output: [1, 2, 3, 4] (also modified)

# Now checking identity, they point to the same object
print(list1 is list2) # Output: True (same memory location)
```

Control Flow (If/Else)

- if, elif, and else are used to control the flow of execution
- Example:

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

Loops (For/While)

- **For Loops** are used for iterating over sequences
- **While Loops** run as long as a condition is True
- Example (For Loop):

```
for i in range(5):  
    print(i)
```

- Example (While Loop):

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

Functions in Python

- Functions allow you to encapsulate logic into reusable blocks of code
- Functions can have parameters (input) and return values (output)
- Example:

```
def add(a, b):  
    return a + b  
  
result = add(2, 3)  
print(result)  # Output: 5
```


Functions in Python (Default Parameters)

- Functions can also have **default parameters**, which provide default values if not specified by the caller
- Example :

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

# Using the default parameter for greeting
message1 = greet("John")
print(message1)  # Output: Hello, John!

# Providing a custom greeting
message2 = greet("John", "Hi")
print(message2)  # Output: Hi, John!
```

Lambda Functions in Python

- **Lambda functions** are anonymous functions defined with the `lambda` keyword
- They can take any number of arguments but only have one expression

```
# Lambda function Syntax
lambda arguments: expression
```

- Example:

```
# Lambda to add two numbers
add = lambda x, y: x + y
result = add(2, 3)
print(result) # Output: 5
```

Lambda Functions in Python

- Lambda Functions are useful as argument functions in more complex methods
- Example with sorted:

```
# Sort a list of tuples by the second element
pairs = [(1, 'b'), (2, 'a'), (3, 'c')]
sorted_pairs = sorted(pairs, key=lambda x: x[1])
print(sorted_pairs) # Output: [(2,'a'), (1,'b'), (3,'c')]
```

Introduction to NumPy

- **NumPy** is a powerful Python library for numerical computing
- It provides support for multi-dimensional arrays and mathematical functions
- Arrays in NumPy are more efficient than Python lists due to fixed data types
- Example of usage:

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr)    # Output: [1 2 3 4]
```

Array Indexing and Slicing

- NumPy arrays can be indexed and sliced

```
# Indexing

arr = np.array([10, 20, 30, 40])
print(arr[1])    # Output: 20

matrix = np.array([[1, 2], [3, 4], [5, 6]])
print(matrix[1, 1]) # Output: 4 (row 1, column 1)

# Slicing

arr = np.array([10, 20, 30, 40])
print(arr[1:3])   # Output: [20 30]

print(matrix[:, 1]) # Output: [2 4 6] (all rows, column 1)
```

Implicit Casting in NumPy

- NumPy will automatically cast data types to the most suitable type during operations
- Example:

```
arr = np.array([1, 2, 3])  
float_arr = arr + 0.5  
print(float_arr)  # Output: [1.5 2.5 3.5]
```

- Operations involving arrays of different types result in type promotion
- Explicit casting using `astype()`:

```
int_arr = float_arr.astype(int)  
print(int_arr)  # Output: [1 2 3]
```

- Be careful to potential data loss when converting between types!

Matrix Operations in NumPy

NumPy supports basic matrix operations like addition, subtraction, and dot products

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

# Element-wise operations
print(a + b)  # Output: [[ 6  8] [10 12]]
print(a * b)  # Output: [[ 5 12] [21 32]]

# Matrix multiplication
dot_product = np.dot(a, b)
print(dot_product)  # Output: [[19 22] [43 50]]

# Transpose of a Matrix
print(a.T)  # Output: [[1 3] [2 4]]
```

Array Manipulation Functions

NumPy allows reshaping, stacking, and splitting

```
arr = np.array([1, 2, 3, 4, 5, 6])
a = np.array([1, 2])
b = np.array([3, 4])

#Reshaping-1
reshaped = arr.reshape((2, 3))
print(reshaped)    # Output: [[1 2 3] [4 5 6]]

# Reshaping-2
a_reshaped = np.reshape(a,(2,-1)) # Output: array([[1],[2]])
```


Array Manipulation Functions

```
# Vertical stacking
v_stack = np.vstack((a, b))
print(v_stack)  # Output: [[1 2] [3 4]]

# Horizontal stack
h_stack = np.hstack((a, b))
print(h_stack)  # Output: [1 2 3 4]

split_arr = np.split(arr, 3)
print(split_arr)
# Output: [array([1,2]), array([3,4]), array([5,6])]
```

- **Broadcasting** allows NumPy to perform operations on arrays of different dimensions
- Shape of the smaller array is expanded to match the dimension of the largest one
- **Broadcasting Rules:**
 - ▶ If the arrays have a different number of dimensions, pad the shape of the smaller array with ones on the left side
 - ▶ Compare the shapes element-wise. The sizes must be equal or one of them must be 1
 - ▶ If the sizes do not match and neither is 1, an error is raised

NumPy Broadcasting

```
# Create a 1D array of shape (3,)
array_1d = np.array([1, 2, 3]) # Shape: (3,)

# Create a 2D array of shape (2, 3)
array_2d = np.array([[4, 5, 6],
                     [7, 8, 9]]) # Shape: (2, 3)

# Attempt element-wise multiplication (broadcasting)
result = array_1d * array_2d
print("Result of element-wise multiplication:\n", result)
print("Result shape:", result.shape) # Display the shape

Result of element-wise multiplication:
[[ 4 10 18]
 [ 7 16 27]]
Result shape: (2, 3)
```

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y) # Create the plot

# Add labels and title
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.title('Sine Wave')

# Show the plot
plt.show()
```

Plot Customization in Matplotlib

Matplotlib allows for extensive customization of plots

```
# Customize the line style, color, and markers
plt.plot(x, y, color='red', linestyle='--', marker='o')

# Adding a grid
plt.grid(True)

# Adding a legend
plt.legend(['Sine Wave'])

plt.show()
```

Subplots in Matplotlib

Subplots allow multiple plots to be displayed in the same figure, which is useful for comparative visualizations.

```
# Create subplots (2 rows, 1 column)
fig, ax = plt.subplots(2, 1)

# First plot
ax[0].plot(x, np.sin(x))
ax[0].set_title('Sine Wave')

# Second plot
ax[1].plot(x, np.cos(x))
ax[1].set_title('Cosine Wave')

plt.tight_layout() # Adjust spacing between plots
plt.show()
```

Introduction to Object-Oriented Programming (OOP)

- **Object-Oriented Programming (OOP)** is a programming paradigm based on the concept of *objects*
- Objects contain both data (attributes) and functions (methods) that operate on the data
- Encapsulation, abstraction, inheritance and polymorphism
- Python is mainly an object oriented language



Classes and Objects

- A **Class** is a blueprint for creating objects
- An **Object** is an instance of a class. Each object can have attributes and methods.
- Example: Defining a class and creating an object

```
class Dog:
    def __init__(self, name, breed):
        self.name = name    # Attribute
        self.breed = breed  # Attribute

    def bark(self):    # Method
        return f"{self.name} says Woof!"

# Create an object (instance) of the Dog class
dog1 = Dog("Argo", "Golden Retriever")

print(dog1.bark())    # Output: Buddy says Woof!
```


Constructor Method in Python

- A **constructor** is the method called when a new instance of a class is created
- The constructor method is defined by the `__init__` function in Python
- Constructors can take arguments to initialize object attributes

```
class Car:
    def __init__(self, make, model, year):  # Constructor
        self.make = make  # Initializing the make attribute
        self.model = model  # Initializing the model attribute
        self.year = year  # Initializing the year attribute

    def display_info(self):
        return f"{self.year} {self.make} {self.model}"

# Creating an object of the Car class
car1 = Car("Toyota", "Camry", 2021)

print(car1.display_info())  # Output: 2021 Toyota Camry
```

Inheritance

- **Inheritance** allows a class to inherit attributes and methods from another class.
- This promotes code reusability and a hierarchical class structure.

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass # Placeholder for subclass implementation

class Dog(Animal):
    def speak(self):
        return f"{self.name} barks."

class Cat(Animal):
    def speak(self):
        return f"{self.name} meows."
```

Conclusion

Python is a readable and versatile language

- High-level syntax
- Interpreted language
- Strong community support

Object-oriented structure

- Efficient coding, reusable code
- Organized, encapsulated code

Python for Control System Design

- Easy to model and simulate control systems (NumPy, SciPy)
- Can be compiled into real-time executables (PyPy, Cython)
- Widely used for solving optimization problems (SciPy.optimize, CVXPY, Pyomo, ...)

